La librería Matplotlib en Python

Matplotlib es una librería de Python especializada en la creación de gráficos en dos dimensiones.



Permite crear y personalizar los tipos de gráficos más comunes, entre ellos, histogramas y diagramas de barras, sectores, caja, violín, dispersión o puntos, líneas, áreas o sectores, contornos, mapas de color y combinaciones de todos ellos.

En la siguiente galería de gráficos pueden apreciarse todos los tipos de gráficos que pueden crearse con esta librería.
Previamente hay que instalar en el terminal del equipo (CMD, Powershell o Linux como administrador) los siguientes comandos:

- pip install matplotlib

Para importar la librería Matplotlib y sus componentes debemos adjuntar al principio del programa, función o procedimiento que usemos la siguiente línea:

- import matplotlib.pyplot as plt

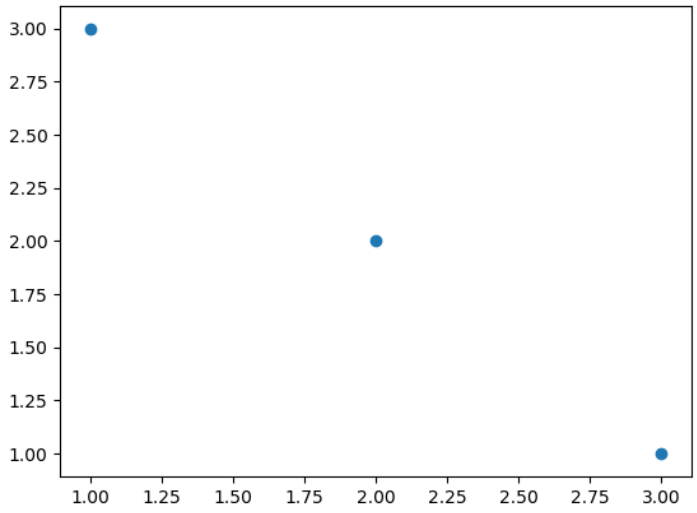
Creación de gráficos con matplotlib

Para crear un gráfico con matplotlib es habitual seguir los siguientes pasos:

1. **Importar el módulo pyplot.**
2. **Definir la figura que contendrá el gráfico,** que es la ventana o página donde se dibujará y los ejes sobre los que se dibujarán los datos. Para ello se utiliza la función subplots().
3. **Dibujar los datos sobre los ejes.** Para ello se utilizan distintas funciones dependiendo del tipo de gráfico que se quiera.
4. **Personalizar el gráfico.** Para ello existen multitud de funciones que permiten añadir un título, una leyenda, una rejilla, cambiar colores o personalizar los ejes.
5. **Guardar el gráfico**. Para ello se utiliza la función **savefig().**
6. **Mostrar el gráfico.** Para ello se utiliza la función **show().**

Ejemplo de uso básico de la librería:

```
# Importar el módulo pyplot con el alias plt
import matplotlib.pyplot as plt
# Crear la figura y los ejes
fig, ax = plt.subplots()
# Dibujar puntos
ax.scatter(x = [1, 2, 3], y = [3, 2, 1])
# Guardar el gráfico en formato png
plt.savefig('diagrama-dispersion.png')
# Mostrar el gráfico
plt.show()
```

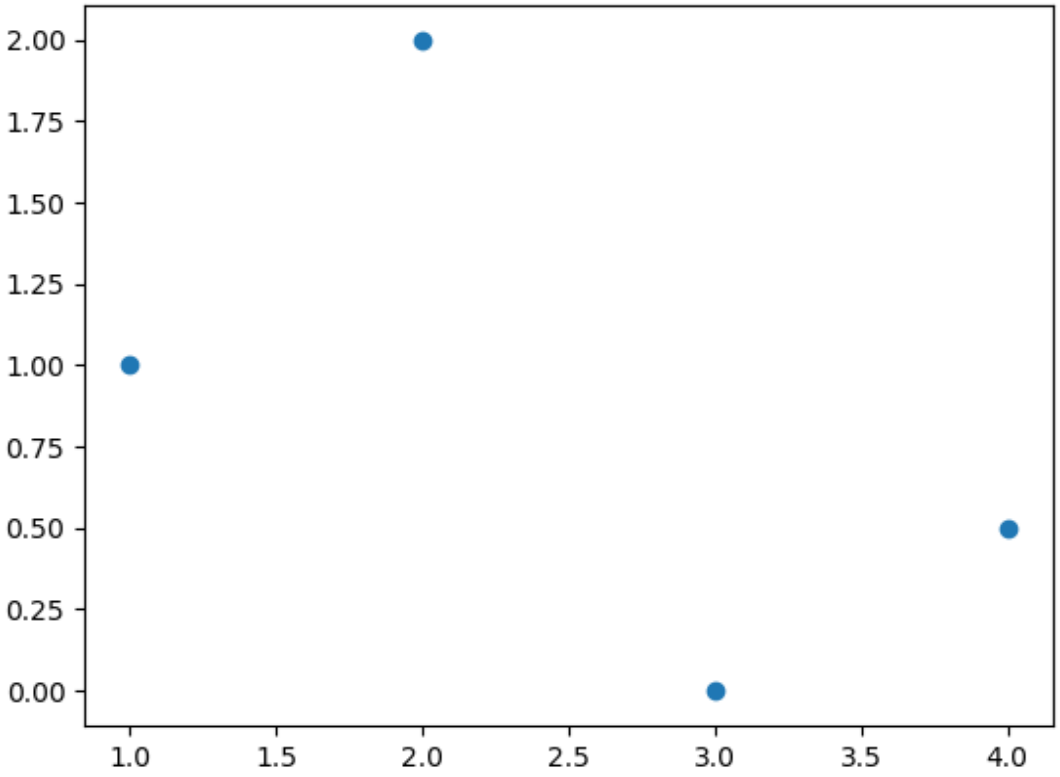


Diagramas de dispersión o puntos

Un diagrama de dispersión, gráfica de dispersión, gráfico de burbujas o gráfico de puntos es un tipo de diagrama matemático que utiliza las coordenadas cartesianas para mostrar los valores de dos variables para un conjunto de datos.

- **scatter(x, y):** Dibuja un diagrama de puntos con las coordenadas de la lista x en el eje X y las coordenadas de la lista y en el eje Y.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.scatter([1, 2, 3, 4], [1, 2, 0, 0.5])
plt.show()
```

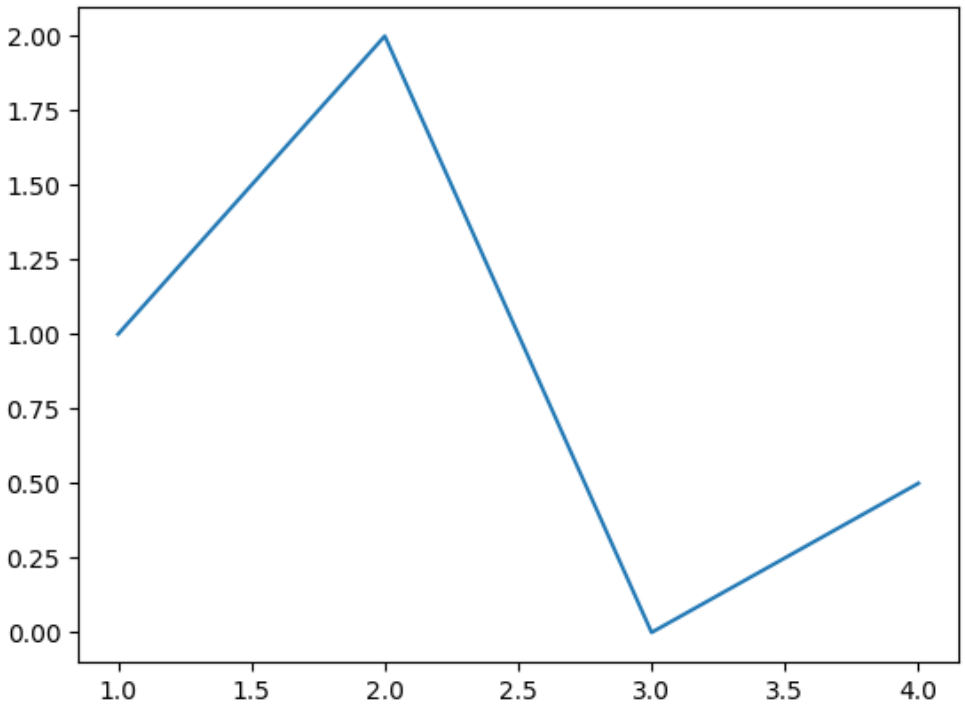


Diagramas de líneas

Un diagrama de líneas representa una serie de puntos de datos en un gráfico y los conecta a líneas. Un diagrama de líneas es especialmente útil cuando se muestran líneas de tendencia con diferencias sutiles, o con líneas de datos que se cruzan.

- **plot(x, y):** Dibuja un polígono con los vértices dados por las coordenadas de la lista x en el eje X y las coordenadas de la lista y en el eje Y.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [1, 2, 0, 0.5])
plt.show()
```

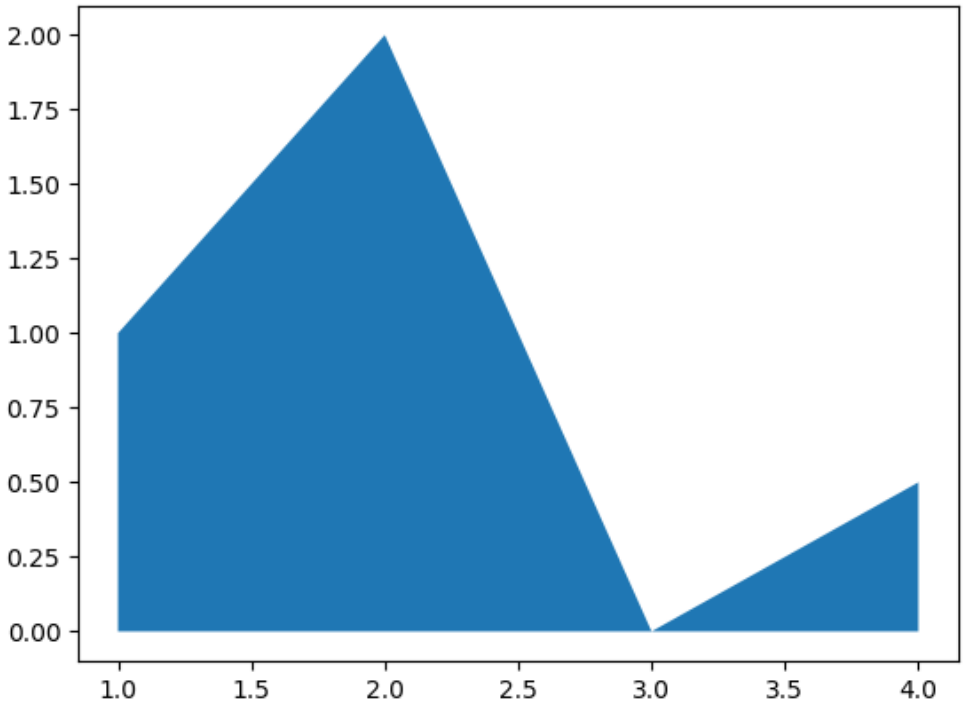


Diagramas de áreas

Es similar al diagrama de líneas, salvo que se rellena con color u otras marcas la parte inferior del polígono formado.

- **fill_between(x, y):** Dibuja el area bajo el polígono con los vértices dados por las coordenadas de la lista x en el eje X y las coordenadas de la lista y en el eje Y.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.fill_between([1, 2, 3, 4], [1, 2, 0, 0.5])
plt.show()
```

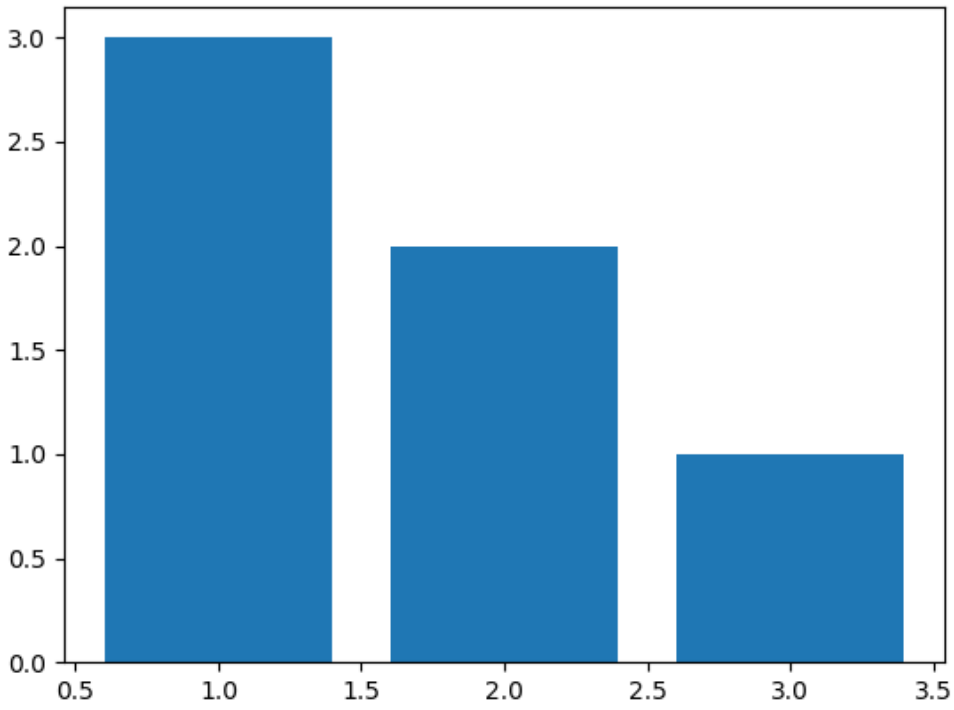


Diagramas de barras verticales

Un diagrama de barras, gráfico de barras o gráfico de columnas es una forma de representar gráficamente un conjunto de datos o valores mediante barras rectangulares de longitud proporcional a los valores representados. Los gráficos de barras pueden ser usados para comparar cantidades de una variable en diferentes momentos o diferentes variables para el mismo momento.

- **bar(x, y):** Dibuja un diagrama de barras verticales donde x es una lista con la posición de las barras en el eje X, e y es una lista con la altura de las barras en el eje Y.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.bar([1, 2, 3], [3, 2, 1])
plt.show()
```

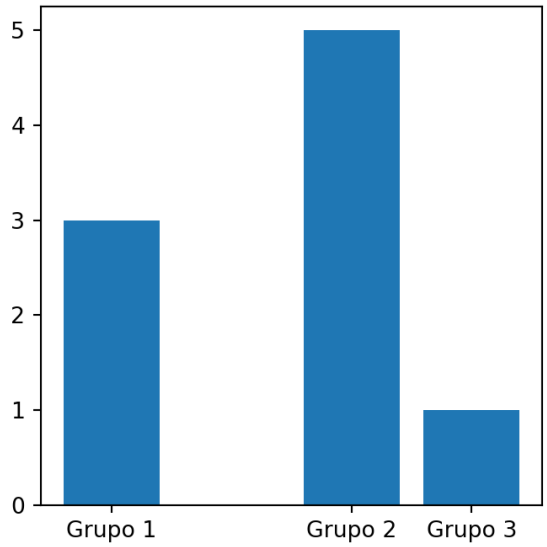


Elementos a añadir a un diagrama de barras

1. Etiquetas de las barras

En el ejemplo anterior también puedes agregar las etiquetas con el argumento tick_label, como en el ejemplo siguiente.

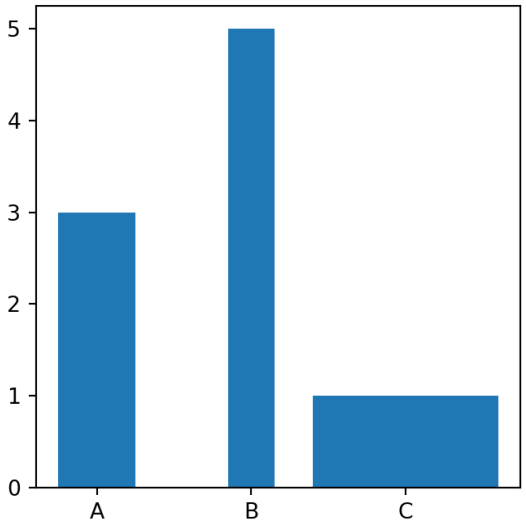
```
        # Datos
x = [1, 3, 4]
y = [3, 5, 1]
etiquetas = ["Grupo 1", "Grupo 2", "Grupo 3"]
        # Gráfico de barras
fig, ax = plt.subplots()
ax.bar(x = x, height = y, tick_label = etiquetas)
plt.show()
```



2. Grosor de las barras

Por defecto, el grosor de las barras es 0.8. Sin embargo, puedes pasar un número mediante width para cambiar el grosor de todas las barras o pasar un array especificando el grosor para cada una de las barras, como en el bloque de código siguiente:

```
        # Datos
x = ["A", "B", "C"]
y = [3, 5, 1]
grosor = [0.5, 0.3, 1.2]
        # Gráfico de barras
fig, ax = plt.subplots()
ax.bar(x = x, height = y, width = grosor)
plt.show()
```

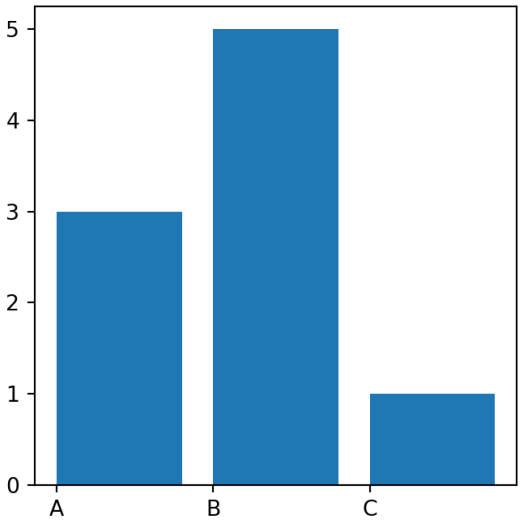


3. Alineación de barras

La alineación de las barras respecto a los valores del eje X es **"center"** por defecto, pero también puedes utilizar **align = "edge"** para alinear los bordes de las barras a las coordenadas de X.

```
# Datos
x = ["A", "B", "C"]
y = [3, 5, 1]

# Gráfico de barras
fig, ax = plt.subplots()
ax.bar(x = x, height = y, align = "edge")
plt.show()
```

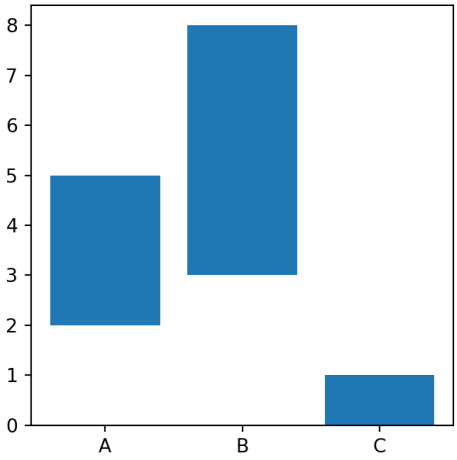


4. Coordenadas de inicio de las barras en el eje Y

Las barras empiezan por defecto **en Y = 0,** pero el argumento bottom permite sobrescribir las coordenadas base para una o cada una de las barras.

```
# Datos
x = ["A", "B", "C"]
y = [3, 5, 1]

# Gráfico de barras
fig, ax = plt.subplots()
ax.bar(x = x, height = y, bottom = [2, 3, 0])
plt.show()
```

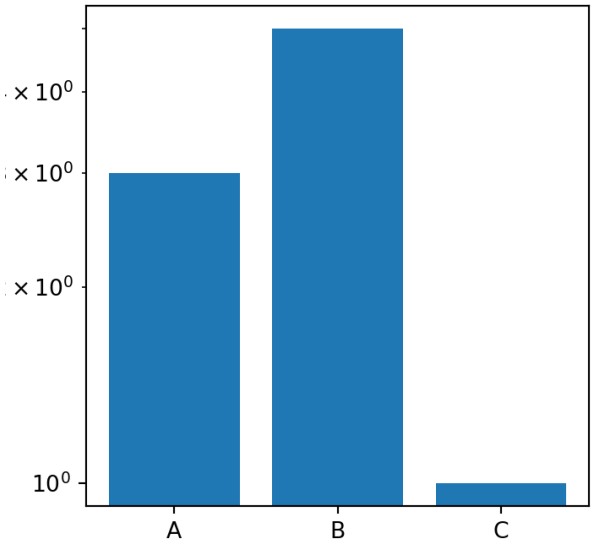


5. Eje Y en escala logarítmica

Por último, ten en cuenta que es posible dibujar el eje Y en escala logarítimica estableciendo log = True.

```
# Datos
x = ["A", "B", "C"]
y = [3, 5, 1]

# Gráfico de barras
fig, ax = plt.subplots()
ax.bar(x = x, height = y, log = True)
plt.show()
```

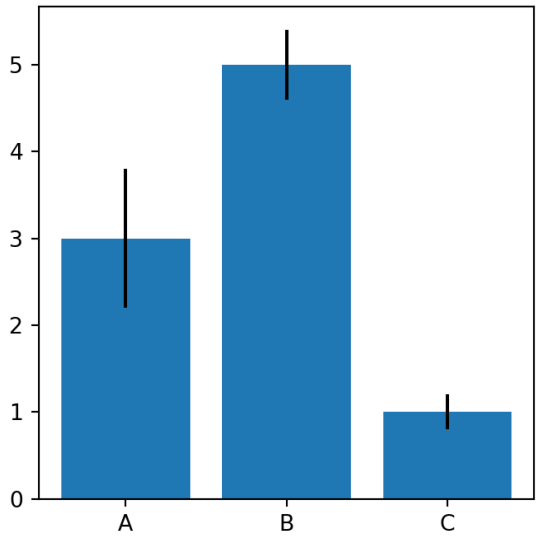


6. Diagrama de barras con barras de error

Tanto **bar como barh** proporcionan un argumento para agregar barras de error tanto verticales como horizontales llamados **yerr y xerr.**

```
# Datos
x = ["A", "B", "C"]
y = [3, 5, 1]
error = [0.8, 0.4, 0.2]

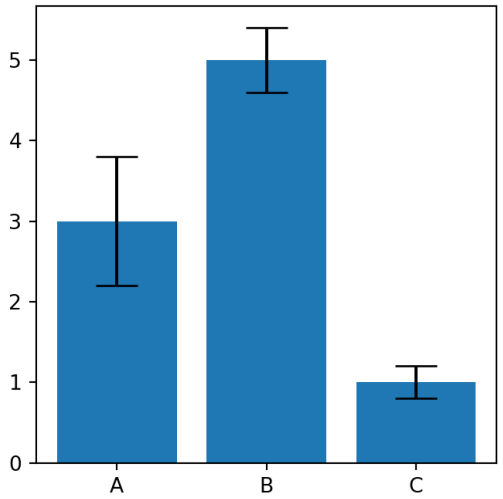
# Gráfico de barras
fig, ax = plt.subplots()
# Se usa yerr si el gráfico es vertical
# Utiliza xerr cuando uses barh
ax.bar(x = x, height = y, yerr = error)
plt.show()
```



Ten en cuenta que también puedes agregar las barras horizontales estableciendo un tamaño adecuado con **capsize.**

```
        # Datos
x = ["A", "B", "C"]
y = [3, 5, 1]
error = [0.8, 0.4, 0.2]

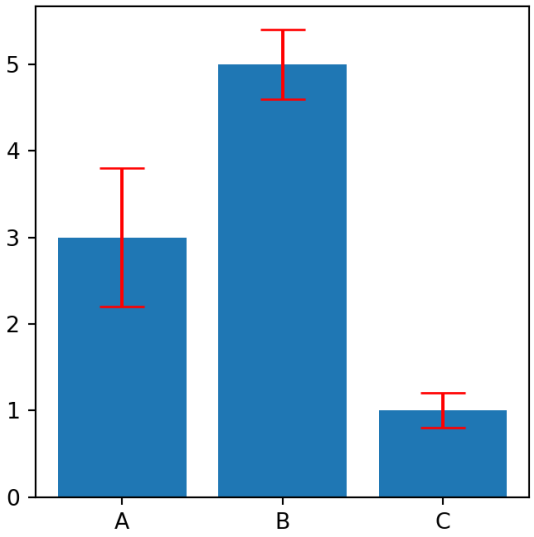

        # Gráfico de barras
fig, ax = plt.subplots()
ax.bar(x = x, height = y, yerr = error, capsize = 10)
plt.show()
```



Por último, también puedes modificar el color de las barras con ecolor y en el siguiente bloque de código las ponemos de color rojo. Utiliza el argumento error_kw para pasar un diccionario con personalizaciones adicionales.

```
        # Datos
x = ["A", "B", "C"]
y = [3, 5, 1]
error = [0.8, 0.4, 0.2]


        # Gráfico de barras
fig, ax = plt.subplots()
ax.bar(x = x, height = y, yerr = error,
                capsize = 10, ecolor = "r")
plt.show()
```



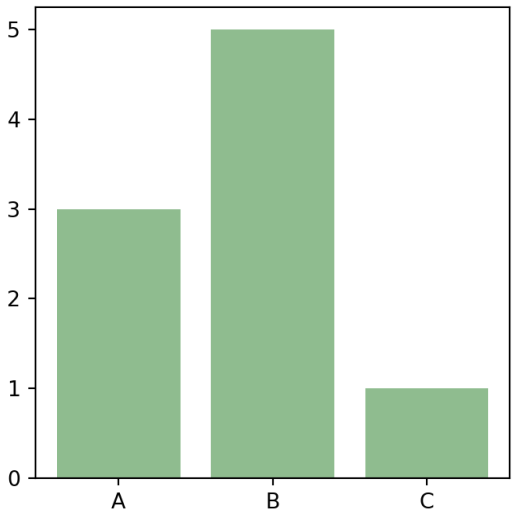
Personalización del color

1. Color de fondo

El color de fondo azul de los dibujos que viene por defecto se puede personalizar con el argumento **color.** Por una parte, puedes pasar un nuevo color al argumento para cambiar el color de todas los elementos a la vez.

```
        # Datos
x = ["A", "B", "C"]
y = [3, 5, 1]

        # Gráfico de barras
fig, ax = plt.subplots()
ax.bar(x = x, height = y,
                color = "darkseagreen")
plt.show()
```

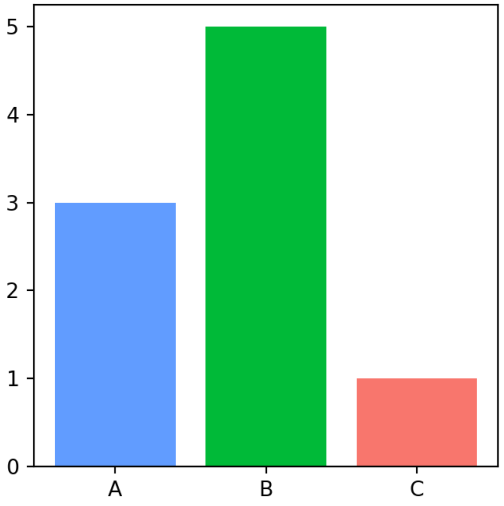


2. Color de fondo por grupo

Por otra parte, si prefieres establecer un color para cada barra puedes pasar un array al argumento de la función con los colores que quieras.

```
        # Datos
x = ["A", "B", "C"]
y = [3, 5, 1]
colores = ["#619cff", "#00ba38", "#f8766d"]

        # Gráfico de barras
fig, ax = plt.subplots()
ax.bar(x = x, height = y, color = colores)
#plt.show()
```

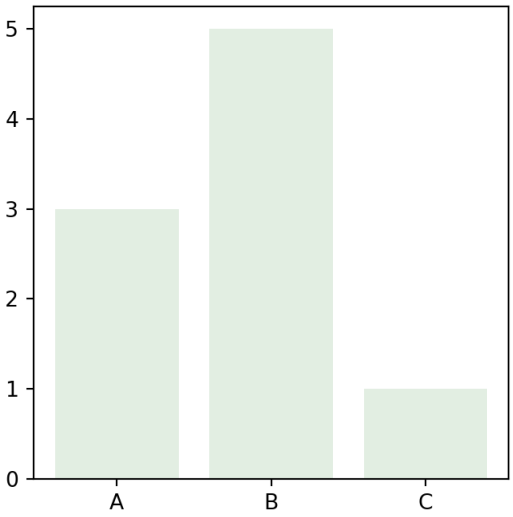


3. Transparencia

Ten en cuenta que el argumento **alpha** permite cambiar la transparencia del color de fondo de las barras pasando un número entre 0 y 1.

```
        # Datos
x = ["A", "B", "C"]
y = [3, 5, 1]

        # Gráfico de barras
fig, ax = plt.subplots()
ax.bar(x = x, height = y,
        color = "darkseagreen", alpha = 0.25)
plt.show()
```

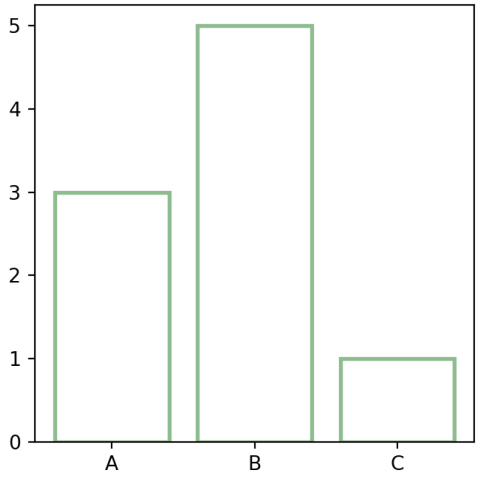


4. Color del borde

Por último, también puedes personalizar el color del borde de las barras con **edgecolor,** tanto para todas las barras como para cada una de ellas. Ten en cuenta que **linewidth** te permitirá cambiar el grosor de los bordes.

```
        # Datos
x = ["A", "B", "C"]
y = [3, 5, 1]

        # Gráfico de barras
fig, ax = plt.subplots()
ax.bar(x = x, height = y, color = "w",
     edgecolor = "darkseagreen", linewidth = 2)
plt.show()
```



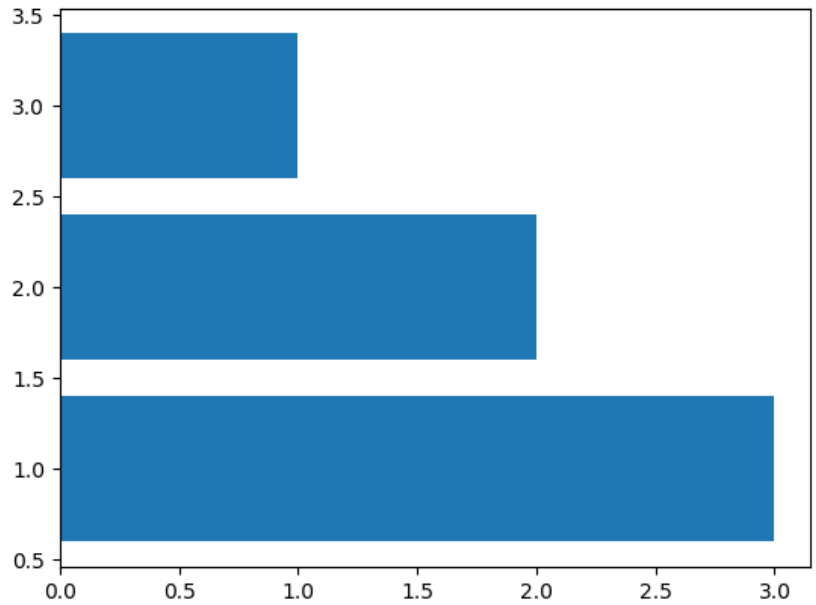
Diagramas de barras horizontales

La función anterior permite crear diagramas de barras verticales en Python, pero si quieres crear uno horizontal tendrás que utilizar la función **barh en lugar de bar.** Esta función es muy parecida a la anterior, con pequeños cambios en los argumentos. Por ejemplo, en lugar de utilizar el argumento **height** tendrás que utilizar **width,** tal y como se muestra a continuación.

Se puede interpretar como una visión traspuesta (intercambiada) de un diagrama de líneas.

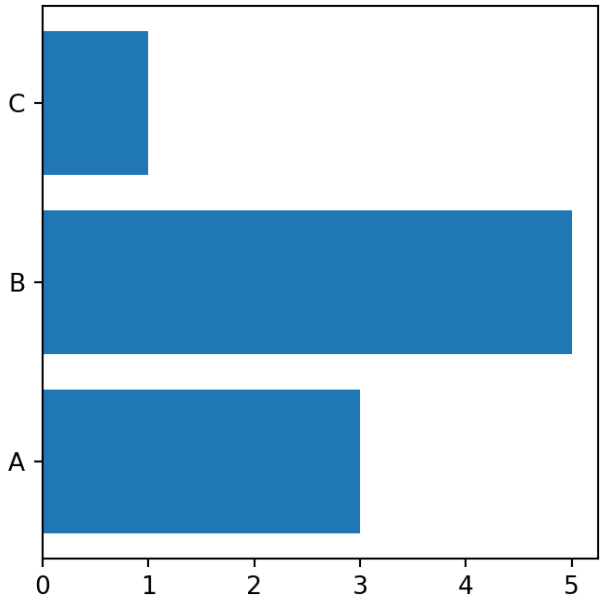
- **barh(x, y):** Dibuja un diagrama de barras horizontales donde x es una lista con la posición de las barras en el eje Y, e y es una lista con la longitud de las barras en el eje X.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.barh([1, 2, 3], [3, 2, 1])
plt.show()
```



```
# Datos
x = ["A", "B", "C"]
y = [3, 5, 1]

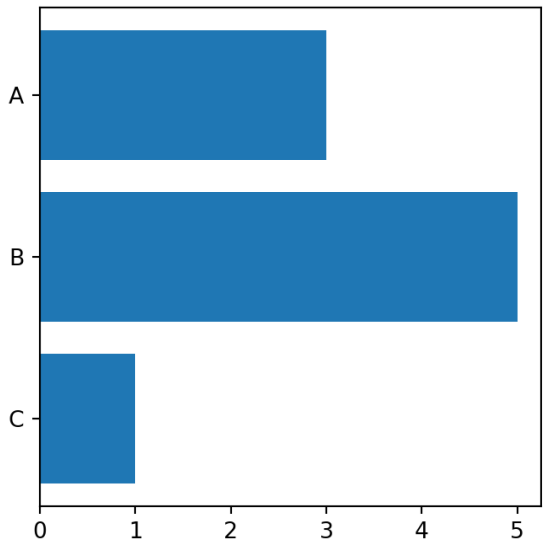
# Gráfico de barras
fig, ax = plt.subplots()
ax.barh(x, width = y)
plt.show()
```



Puede que hayas notado que las categorías no se muestran en el orden del array. Sin embargo, puedes invertir el eje Y con **invert_yaxis(),** de modo que las etiquetas se lean de arriba a abajo.

```
# Datos
x = ["A", "B", "C"]
y = [3, 5, 1]

# Gráfico de barras
fig, ax = plt.subplots()
ax.barh(x, width = y)
ax.invert_yaxis()
plt.show()
```

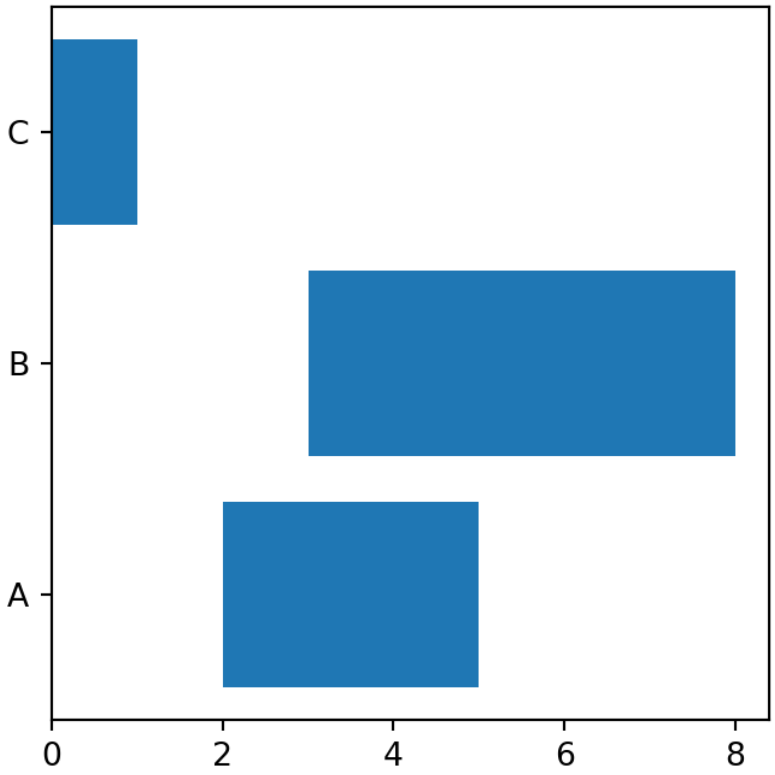


Coordenadas de inicio de las barras en el eje X

Al utilizar barh, el argumento equivalente **a bottom en bar es left,** que permite modificar el inicio de las barras en el eje X.

```
        # Datos
x = ["A", "B", "C"]
y = [3, 5, 1]

        # Gráfico de barras
fig, ax = plt.subplots()
ax.barh(x, width = y, left = [2, 3, 0])
plt.show()
```

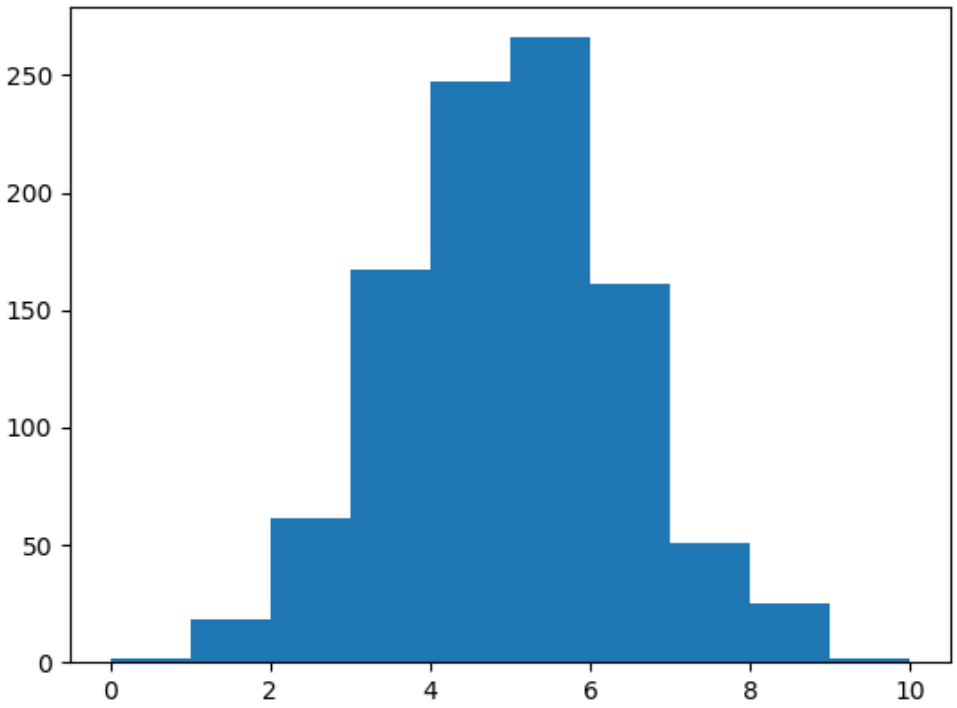


Histogramas

Un histograma es una representación gráfica de una variable en forma de barras, donde la superficie de cada barra es proporcional a la frecuencia de los valores representados.

- **hist(x, bins):** Dibuja un histograma con las frecuencias resultantes de agrupar los datos de la lista x en las clases definidas por la lista bins.

```
import numpy as np
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
x = np.random.normal(5, 1.5, size=1000)
ax.hist(x, np.arange(0, 11))
plt.show()
```

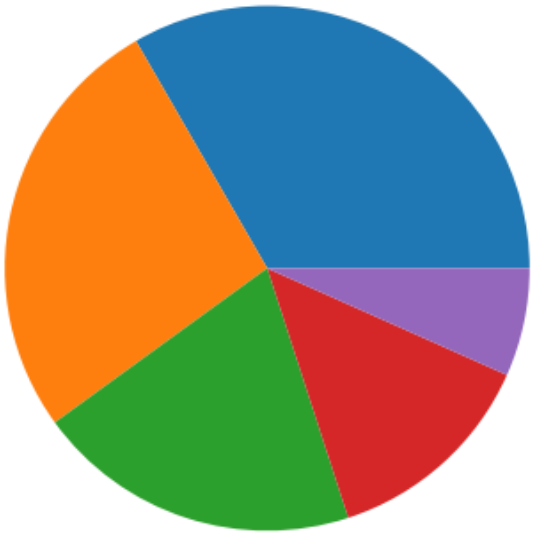


Diagramas de sectores

Un gráfico circular, diagrama sectorial, gráfica circular o gráfica de pastel es un recurso estadístico que se utiliza para representar porcentajes y proporciones. El número de elementos comparados dentro de una gráfica circular suele ser de más de cuatro.

- **pie(x):** Dibuja un diagrama de sectores con las frecuencias de la lista x.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.pie([5, 4, 3, 2, 1])
plt.show()
```



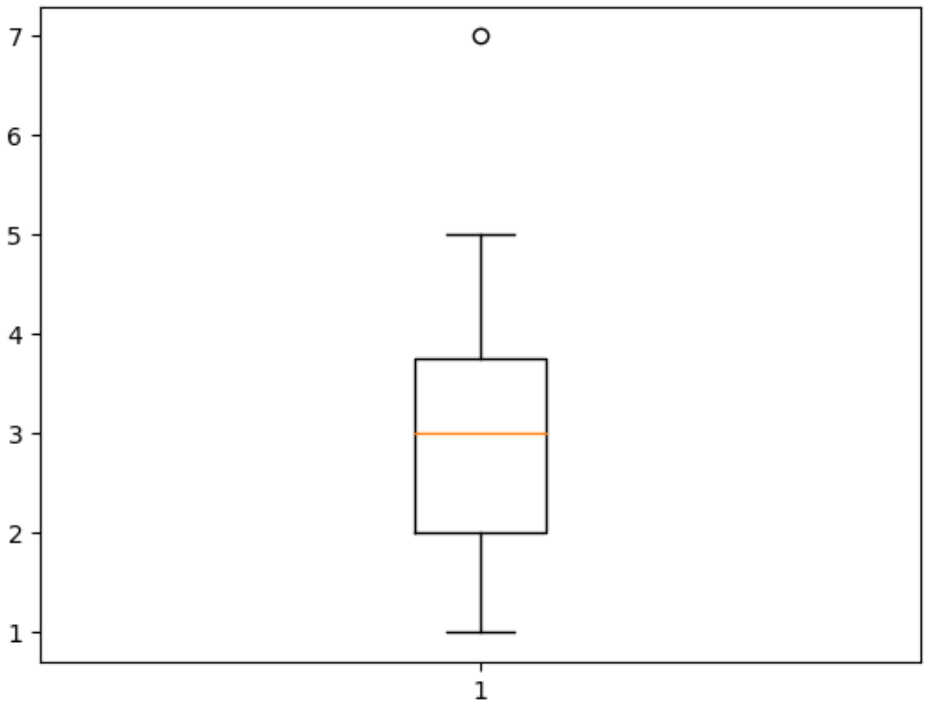
Diagramas de caja y bigotes

Los diagramas de Caja-Bigotes (boxplots o box and whiskers) son una presentación visual que describe varias características importantes, al mismo tiempo, tales como la dispersión y simetría.

Para su realización se representan los tres cuartiles y los valores mínimo y máximo de los datos, sobre un rectángulo, alineado horizontal o verticalmente.

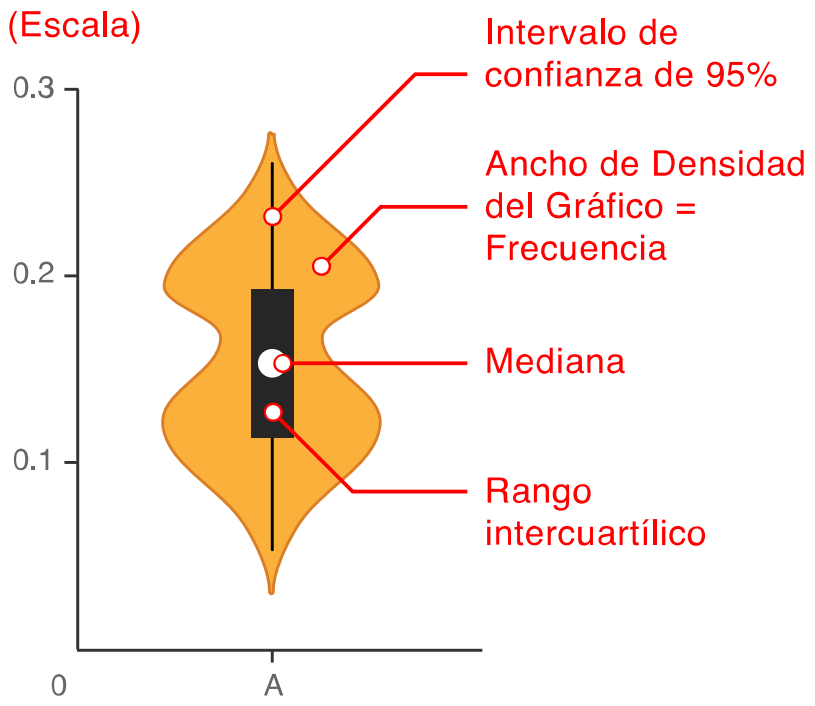
- **boxplot(x):** Dibuja un diagrama de caja y bigotes con los datos de la lista x.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.boxplot([1, 2, 1, 2, 3, 4, 3, 3, 5, 7])
plt.show()
```



Diagramas de violín

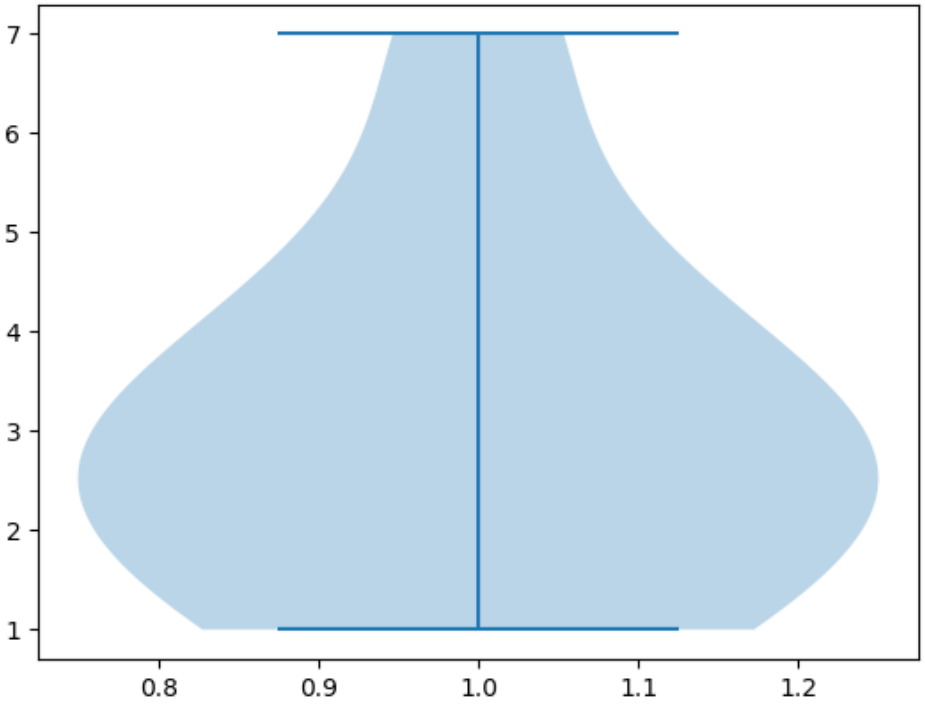
Un diagrama de violín se utiliza para visualizar la distribución de los datos y su densidad de probabilidad.



Este gráfico es una combinación de un diagrama de cajas y bigotes y un diagrama de densidad girado y colocado a cada lado, para mostrar la forma de distribución de los datos. La barra negra gruesa en el centro representa el intervalo intercuartil, la barra negra fina que se extiende desde ella, representa el 95 % de los intervalos de confianza, y el punto blanco es la mediana.

- **violinplot(x):** Dibuja un diagrama de violín con los datos de la lista x.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.violinplot([1, 2, 1, 2, 3, 4, 3, 3, 5, 7])
plt.show()
```



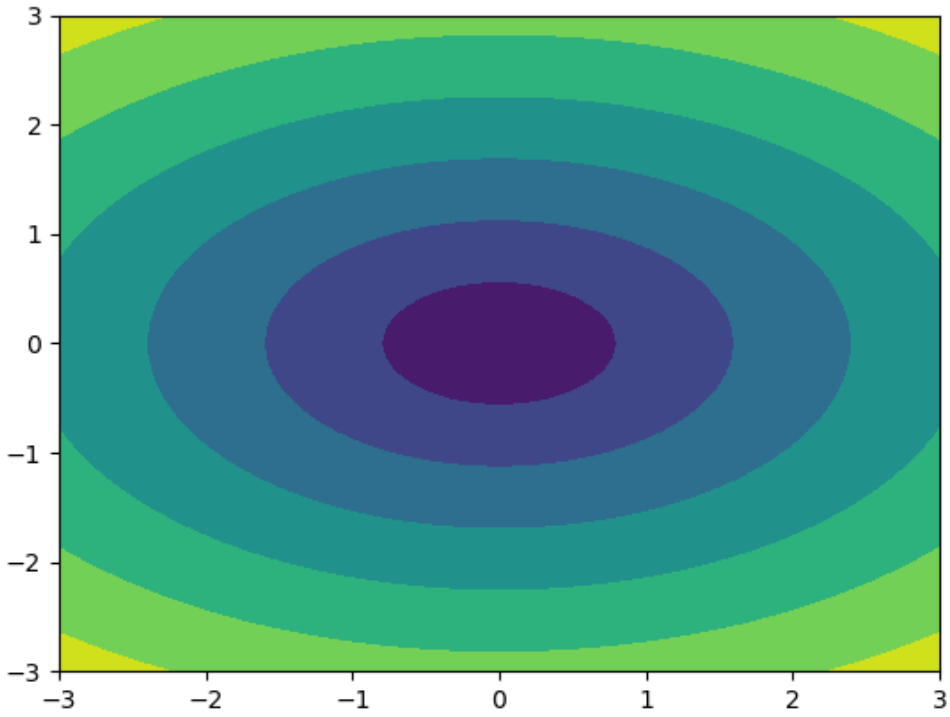
Diagramas de contorno

Una gráfica de contorno representa una superficie 3D trazando líneas que conectan puntos con valores z comunes a lo largo de un corte. Por ejemplo, puede utilizar una gráfica de contorno para visualizar la altura de una superficie en dos o tres dimensiones.

Se usa para explorar la relación potencial entre tres variables. Las gráficas de contorno muestran la relación tridimensional en dos dimensiones, graficando los factores X e Y (predictores) en las escalas X e Y del diagrama, y los valores de respuesta se representan mediante contornos. Una gráfica de contorno es como un mapa topográfico en el que se grafican valores de X, Y y Z, en lugar de longitud, latitud y elevación.

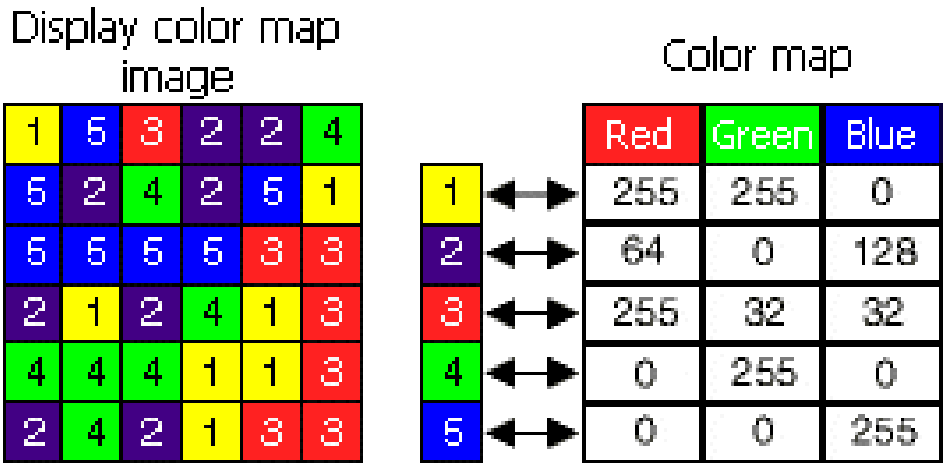
- **contourf(x, y, z):** Dibuja un diagrama de contorno con las curvas de nivel de la superficie dada por los puntos con las coordenadas de las listas x, y y z en los ejes X, Y y Z respectivamente.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
x = np.linspace(-3.0, 3.0, 100)
y = np.linspace(-3.0, 3.0, 100)
x, y = np.meshgrid(x, y)
z = np.sqrt(x**2 + 2*y**2)
ax.contourf(x, y, z)
plt.show()
```



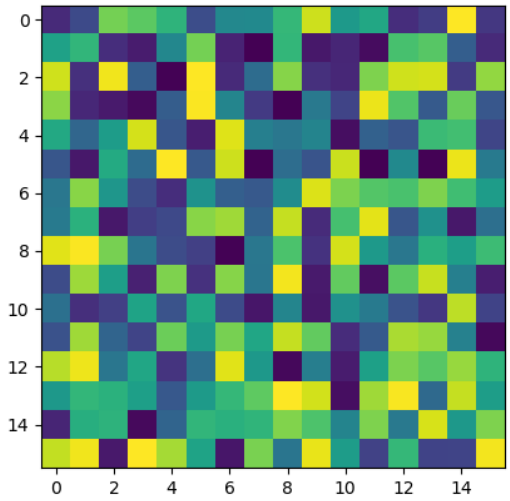
Mapas de color

Un mapa de color es un conjunto de valores que están asociados con colores. Los mapas de color se utilizan para mostrar un ráster de banda única de manera coherente con los mismos colores. Cada valor de píxel está asociado con un color, que se define como un conjunto de valores de rojo, verde y azul (RGB). Como cada valor tiene un color distinto asociado, aparecerá siempre de la misma manera, cada vez que lo abra en un programa que pueda leer un ráster con un mapa de color.



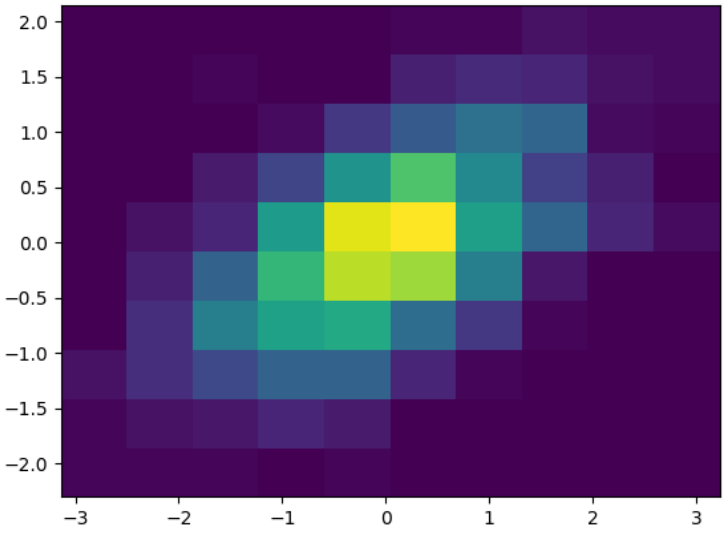
- **imshow(x):** Dibuja un mapa de color a partir de una matriz x (en 2 dimensiones).

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
x = np.random.random((16, 16))
ax.imshow(x)
plt.show()
```



- **hist2d(x, y):** Dibuja un mapa de color que simula un histograma bidimensional, donde los colores de los cuadrados dependen de las frecuencias de las clases de la muestra dada por las listas x e y.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
x, y = np.random.multivariate_normal(mean=[0.0, 0.0], cov=[[1.0, 0.4], [0.4, 0.5]], size=1000).T
ax.hist2d(x, y)
plt.show()
```



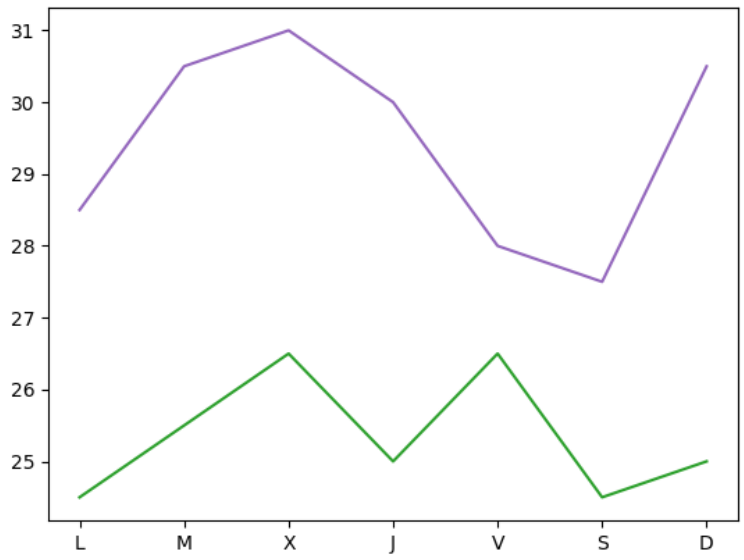
Cambiar el aspecto de los gráficos

Los gráficos creados con Matplotlib son personalizables y puede cambiarse el aspecto de casi todos sus elementos. Los elementos que suelen modificarse más a menudo son **colores, marcadores de puntos, estilo de líneas, títulos, ejes, leyenda y rejilla.**

Colores

Para cambiar el color de los objetos se utiliza el parámetro **color = nombre-color,** donde nombre-color es una cadena con el nombre del color de entre los colores disponibles.

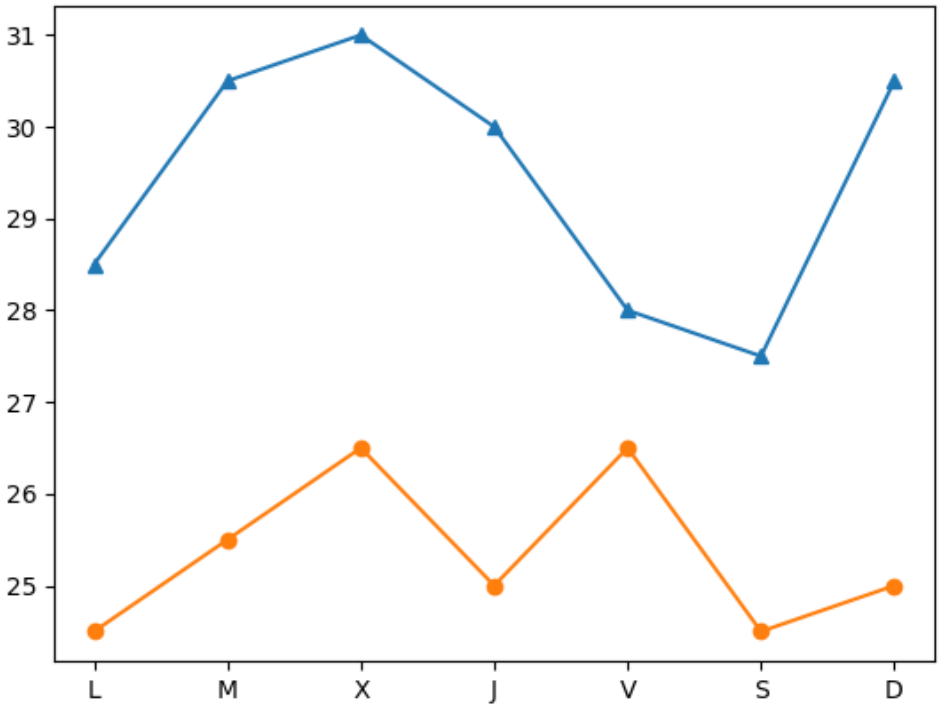
```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
dias = ['L', 'M', 'X', 'J', 'V', 'S', 'D']
temperaturas = {'Madrid':[28.5, 30.5, 31, 30, 28, 27.5, 30.5], 'Barcelona':[24.5, 25.5, 26.5, 25, 26.5, 24.5, 25]}
ax.plot(dias, temperaturas['Madrid'], color = 'tab:purple')
ax.plot(dias, temperaturas['Barcelona'], color = 'tab:green')
plt.show()
```



Marcadores

Para cambiar los marcadores se utiliza el parámetro **marker = nombre-marcador,** donde nombre-marcador es una cadena con el nombre del marcador de entre los marcadores disponibles

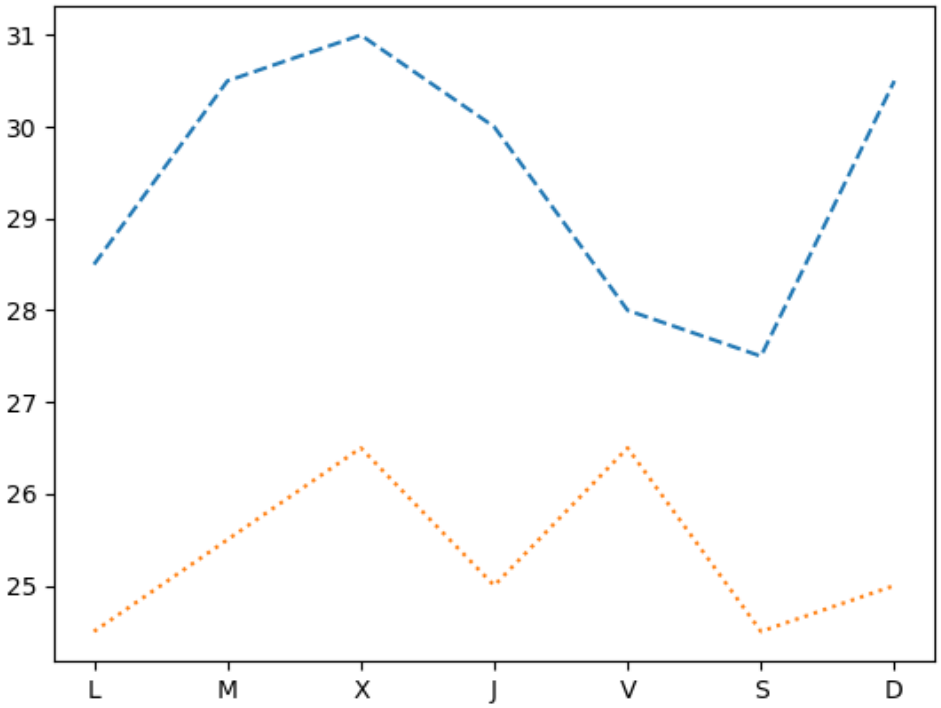
```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
dias = ['L', 'M', 'X', 'J', 'V', 'S', 'D']
temperaturas = {'Madrid':[28.5, 30.5, 31, 30, 28, 27.5, 30.5], 'Barcelona':[24.5, 25.5, 26.5, 25, 26.5, 24.5, 25]}
ax.plot(dias, temperaturas['Madrid'], marker = '^')
ax.plot(dias, temperaturas['Barcelona'], marker = 'o')
plt.show()
```



Líneas

Para cambiar el estilo de las líneas se utiliza el parámetro **linestyle = nombre-estilo**, donde nombre-estilo es una cadena con el nombre del estilo de entre los estilos disponibles

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
dias = ['L', 'M', 'X', 'J', 'V', 'S', 'D']
temperaturas = {'Madrid':[28.5, 30.5, 31, 30, 28, 27.5, 30.5], 'Barcelona':[24.5, 25.5, 26.5, 25, 26.5, 24.5, 25]}
ax.plot(dias, temperaturas['Madrid'], linestyle = 'dashed')
ax.plot(dias, temperaturas['Barcelona'], linestyle = 'dotted')
plt.show()
```

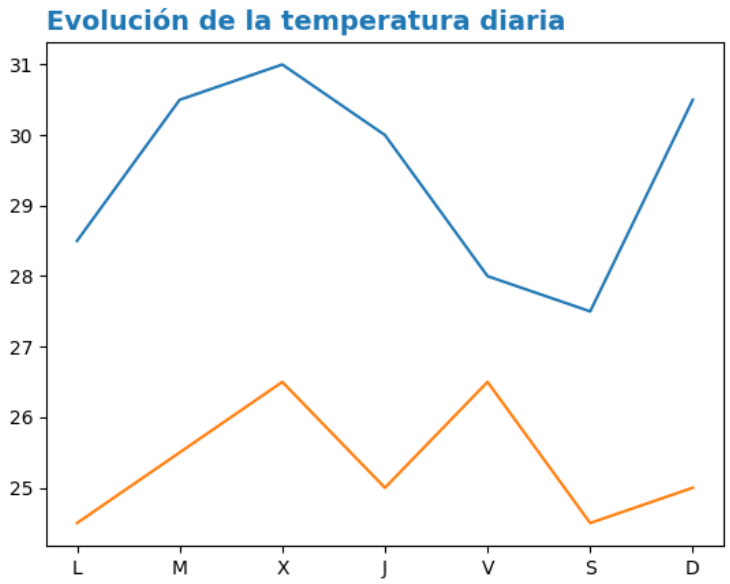


Títulos

Para añadir un título principal al gráfico se utiliza el siguiente método:

- **ax.set_title(titulo, loc=alineacion, fontdict=fuente):** Añade un título con el contenido de la cadena titulo a los ejes ax. El parámetro loc indica la alineación del título, que puede ser 'left' (izquierda), 'center' (centro) o 'right' (derecha), y el parámetro fontdict indica mediante un diccionario las características de la fuente (la el tamaño fontisize, el grosor fontweight o el color color).

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
dias = ['L', 'M', 'X', 'J', 'V', 'S', 'D']
temperaturas = {'Madrid':[28.5, 30.5, 31, 30, 28, 27.5, 30.5], 'Barcelona':[24.5, 25.5, 26.5, 25, 26.5, 24.5, 25]}
ax.plot(dias, temperaturas['Madrid'])
ax.plot(dias, temperaturas['Barcelona'])
ax.set_title('Evolución de la temperatura diaria', loc = "left", fontdict = {'fontsize':14, 'fontweight':'bold', 'color':'tab:blue'})
plt.show()
```



Ejes

Para cambiar el aspecto de los ejes se suelen utilizar los siguientes métodos:

- **ax.set_xlabel(titulo):** Añade un título con el contenido de la cadena título al eje x de ax. Se puede personalizar la alineación y la fuente con los mismos parámetros que para el título principal.

- **ax.set_ylabel(titulo):** Añade un título con el contenido de la cadena título al eje y de ax. Se puede personalizar la alineación y la fuente con los mismos parámetros que para el título principal.

- **ax.set_xlim([limite-inferior, limite-superior]):** Establece los límites que se muestran en el eje x de ax.

- **ax.set_ylim([limite-inferior, limite-superior]):** Establece los límites que se muestran en el eje y de ax.

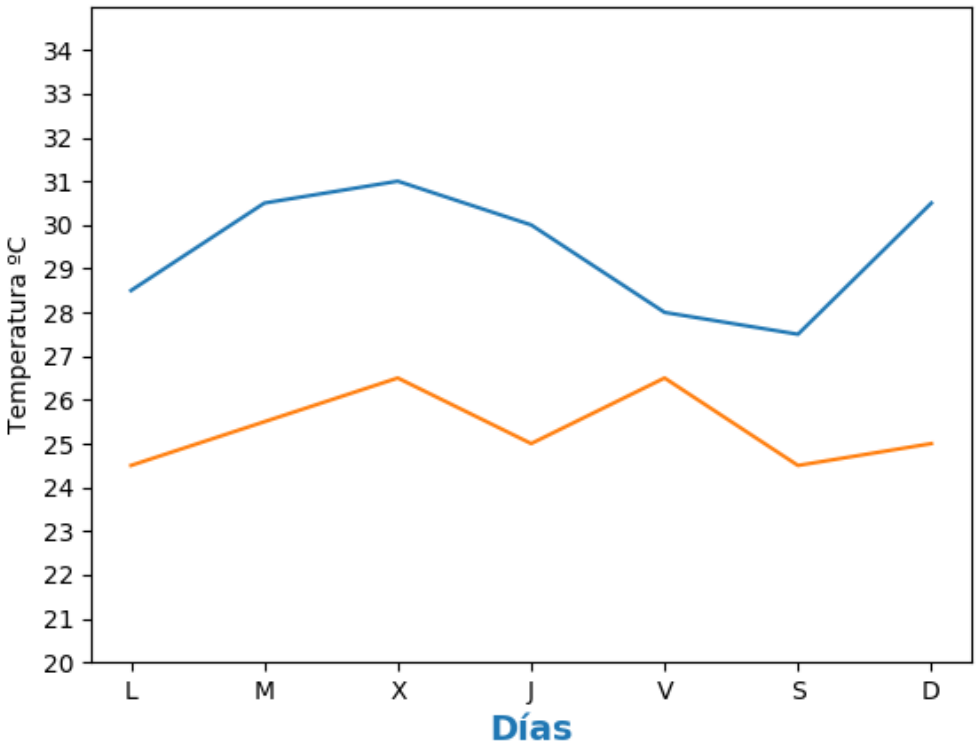
- **ax.set_xticks(marcas):** Dibuja marcas en el eje x de ax en las posiciones indicadas en la lista marcas.

- **ax.set_yticks(marcas):** Dibuja marcas en el eje y de ax en las posiciones indicadas en la lista marcas.

- **ax.set_xscale(escala):** Establece la escala del eje x de ax, donde el parámetro escala puede ser 'linear' (lineal) o 'log' (logarítmica).

- **ax.set_yscale(escala):** Establece la escala del eje y de ax, donde el parámetro escala puede ser 'linear' (lineal) o 'log' (logarítmica).

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
dias = ['L', 'M', 'X', 'J', 'V', 'S', 'D']
temperaturas = {'Madrid':[28.5, 30.5, 31, 30, 28, 27.5, 30.5], 'Barcelona':[24.5, 25.5,
26.5, 25, 26.5, 24.5, 25]}
ax.plot(dias, temperaturas['Madrid'])
ax.plot(dias, temperaturas['Barcelona'])
ax.set_xlabel("Días", fontdict = {'fontsize':14, 'fontweight':'bold', 'color':'tab:blue'})
ax.set_ylabel("Temperatura ºC")
ax.set_ylim([20,35])
ax.set_yticks(range(20, 35))
plt.show()
```



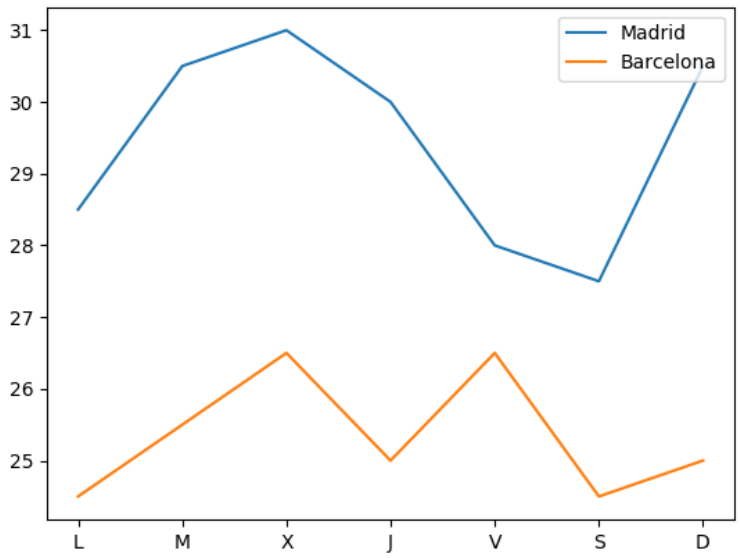
Leyenda

Para añadir una leyenda a un gráfico se utiliza el siguiente método:

- **ax.legend(leyendas, loc = posición):** Dibuja un leyenda en los ejes ax con los nombres indicados en la lista leyendas.

El parámetro loc indica la posición en la que se dibuja la leyenda y puede ser **'upper left'** (arriba izquierda), **'upper center'** (arriba centro), **'upper right'** (arriba derecha), **'center left'** (centro izquierda), **'center'** (centro), **'center right'** (centro derecha), **'lower left'** (abajo izquierda), **'lower center'** (abajo centro), **'lower right'** (abajo derecha). Se puede omitir la lista de leyendas si se indica la leyenda de cada serie en la función que la dibuja mediante el parámetro label.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
dias = ['L', 'M', 'X', 'J', 'V', 'S', 'D']
temperaturas = {'Madrid':[28.5, 30.5, 31, 30, 28, 27.5, 30.5], 'Barcelona':[24.5, 25.5, 26.5, 25, 26.5, 24.5, 25]}
ax.plot(dias, temperaturas['Madrid'], label = 'Madrid')
ax.plot(dias, temperaturas['Barcelona'], label = 'Barcelona')
ax.legend(loc = 'upper right')
plt.show()
```

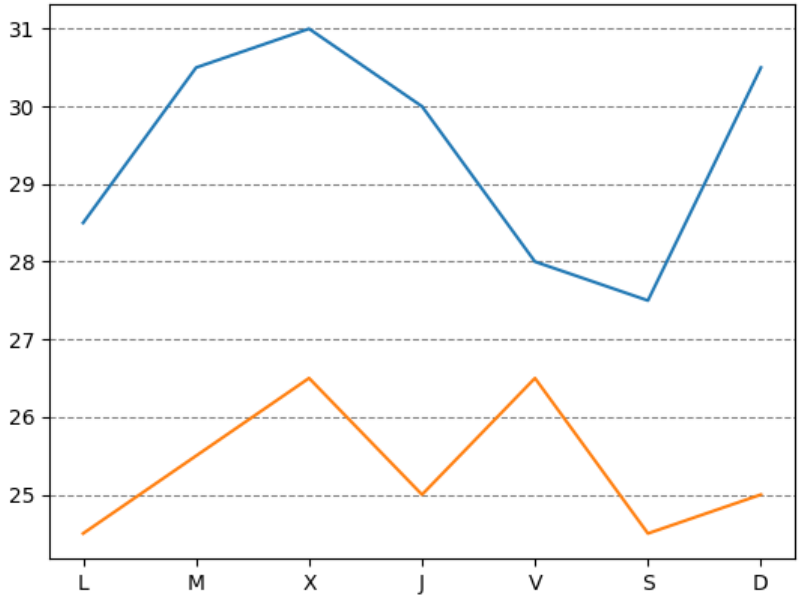


Rejilla

- **ax.grid(axis=ejes, color=color, linestyle=estilo):** Dibuja una rejilla en los ejes de ax.

El parámetro axis indica los ejes sobre los que se dibuja la rejilla y puede ser **'x' (eje x), 'y' (eje y) o 'both' (ambos).** Los parámetros **color y linestyle** establecen el color y el estilo de las líneas de la rejilla, y pueden tomar los mismos valores vistos en los apartados de colores y líneas.

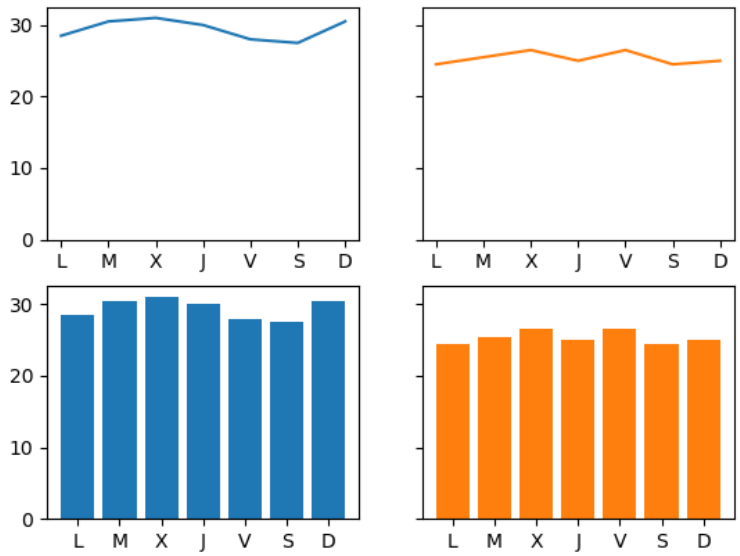
```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
dias = ['L', 'M', 'X', 'J', 'V', 'S', 'D']
temperaturas = {'Madrid':[28.5, 30.5, 31, 30, 28, 27.5, 30.5], 'Barcelona':[24.5, 25.5, 26.5, 25, 26.5, 24.5, 25]}
ax.plot(dias, temperaturas['Madrid'])
ax.plot(dias, temperaturas['Barcelona'])
ax.grid(axis = 'y', color = 'gray', linestyle = 'dashed')
plt.show()
```



Múltiples gráficos

Es posible dibujar varios gráficos en distintos ejes en una misma figura organizados en forma de tabla. Para ello, cuando se inicializa la figura y los ejes, hay que pasarle a la función **subplots** el número de filas y columnas de la tabla que contendrá los gráficos. Con esto los distintos ejes se organizan en un array y se puede acceder a cada uno de ellos a través de sus índices. Si se quiere que los distintos ejes compartan los mismos límites para los ejes se pueden pasar el parámetro **sharex = True para el eje x o el parámetro sharey = True para el eje y.**

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(2, 2, sharey = True)
dias = ['L', 'M', 'X', 'J', 'V', 'S', 'D']
temperaturas = {'Madrid':[28.5, 30.5, 31, 30, 28, 27.5, 30.5], 'Barcelona':[24.5, 25.5, 26.5, 25, 26.5, 24.5, 25]}
ax[0, 0].plot(dias, temperaturas['Madrid'])
ax[0, 1].plot(dias, temperaturas['Barcelona'], color = 'tab:orange')
ax[1, 0].bar(dias, temperaturas['Madrid'])
ax[1, 1].bar(dias, temperaturas['Barcelona'], color = 'tab:orange')
plt.show()
```



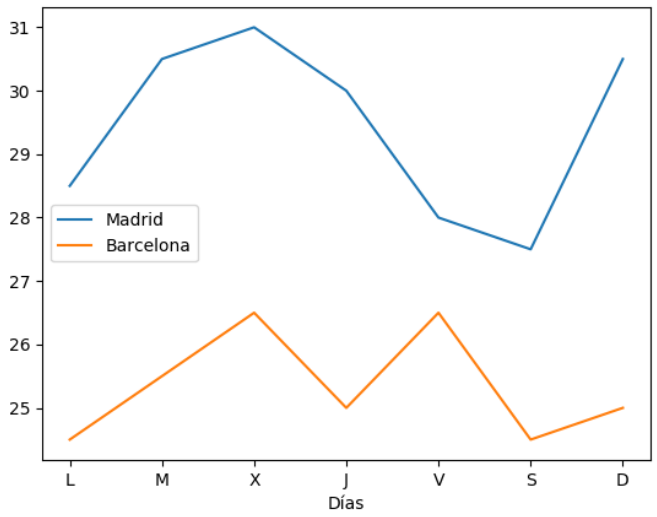
Integración con Pandas

Matplotlib se integra a la perfección con la librería Pandas, permitiendo dibujar gráficos a partir de los datos de las series y DataFrames de Pandas.

- **df.plot(kind=tipo, x=columnax, y=columnay, ax=ejes):** Dibuja un diagrama del tipo indicado por el parámetro kind en los ejes indicados en el parámetro ax, representando en el eje x la columna del parámetro x y en el eje y la columna del parámetro y.

El parámetro kind puede tomar como argumentos **'line' (lineas), 'scatter' (puntos), 'bar' (barras verticales), 'barh' (barras horizontales), 'hist' (histograma), 'box' (cajas), 'density' (densidad), 'area' (area) o 'pie' (sectores).** Es posible pasar otros parámetros para indicar el color, el marcador o el estilo de línea como se ha descrito antes.

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({'Días':['L', 'M', 'X', 'J', 'V', 'S', 'D'],
                   'Madrid':[28.5, 30.5, 31, 30, 28, 27.5, 30.5],
                   'Barcelona':[24.5, 25.5, 26.5, 25, 26.5, 24.5, 25]})
fig, ax = plt.subplots()
df.plot(x = 'Días', y = 'Madrid', ax = ax)
df.plot(x = 'Días', y = 'Barcelona', ax = ax)
plt.show()
```



Si no se indican los parámetros x e y se representa el índice de las filas en el eje x y una serie por cada columna del Dataframe. Las columnas no numéricas se ignoran.

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({'Días':['L', 'M', 'X', 'J', 'V', 'S', 'D'],
                   'Madrid':[28.5, 30.5, 31, 30, 28, 27.5, 30.5],
                   'Barcelona':[24.5, 25.5, 26.5, 25, 26.5, 24.5, 25]})
df = df.set_index('Días')
fig, ax = plt.subplots()
df.plot(ax = ax)
plt.show()
```

